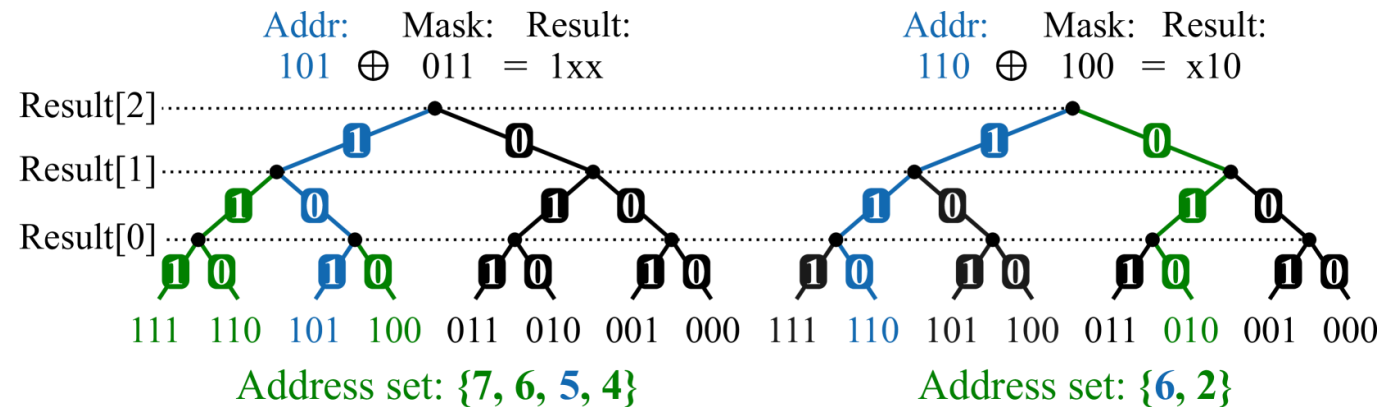


Multicast-Capable AXI Crossbar

- Write multicast with compact address–mask encoding
- AXI WRITE PATH
 - AW Destination selection
 - W Payload replication
 - B Response aggregation

Encoding rule

- $\text{mask}[i] = 0 \rightarrow \text{address}[i] \text{ must match}$
- $\text{Mask}[i] = 1 \rightarrow \text{address}[i] \text{ is don't care}$



ENCODING TRADE-OFF

- Encoding width scales with address width and is independent of destination-set size
- Arbitrary destination sets cannot be represented

Figure 1: Examples of contiguous (left) and strided (right) address sets representable with our encoding, as paths in the binary number tree. Mask bits selectively fork the path of the original address (blue).

Multicast Address Decoding

- **Require every multicast-targetable region**
 - be a power-of-two in size
 - be aligned to an integer multiple of its size

```
mfe.addr = ife.start_addr
mfe.mask = ife.end_addr - ife.start_addr - 1
```

We integrate logic to convert all multicast rules to mask form. Calculating `aw_select` then boils down to:

```
masked_bits      = req.mask | rule.mask
match_bits       = ~(req.addr ^ rule.addr)
aw_select[rule.idx] = &(masked_bits | match_bits)
```

The intersection between the request's and a rule's address sets can be found by resolving the masked bits as:

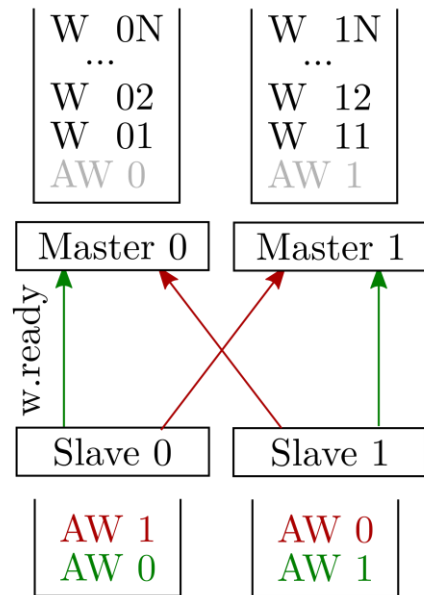
```
out.mask = req.mask & rule.mask
out.addr = (~req.mask & req.addr) | (req.mask &
↪ rule.addr)
```

Interval-form encoding (IFE) to the mask-form encoding (MFE), using the following formulas:

1. Convert each interval-based multicast rule into address-mask form.
2. Compare the request mask with each rule to generate `aw_select`.
3. Resolve the intersection to derive the address subset for each selected slave.

Avoiding Multicast Write Deadlock

- Inconsistent AW ordering creates circular wait



DEADLOCK SCENARIO

Slave 0 accepts: AW0 → AW1

Slave 1 accepts: AW1 → AW0

Solution : Consistent Fixed-Priority Arbitration

- All destination muxes use the same fixed-priority encoder.
- The highest-priority master acquires all target slaves atomically.
- aw.commit is asserted only when all selected muxes are ready.

Multicast Control in axi_mux

- aw.is_mcast: select between unicast and multicast data paths
- aw.commit: prevent deadlocks.
- Both signals are generated by each demux and routed to all muxes.

